ISOMETRIC ZOMBIE SURVIVAL

Comprehensive product, art-direction, simulation, and technical architecture handout

Working concept | Complete indie game | Desktop browser first | WebGPU-first

CENTRAL PROMISE

A cinematic isometric zombie-survival game where the player can physically reshape existing buildings and streets. Every breach, barricade, collapse, obstruction, and improvised route changes sight, sound, navigation, safety, and horde behavior.

How to use this handout

This document is both a creative direction guide and a technical blueprint. It is intended to align design, art, engineering, content production, and coding-agent work before implementation expands. It deliberately distinguishes locked principles, proposed defaults, performance hypotheses, and open decisions.

Label	Meaning
LOCKED	Treat as a project constraint unless the product direction changes.
DEFAULT	Implement this way unless a prototype proves a better option.
BENCHMARK GATE	A measurable target that must be validated on representative hardware.
OPEN	A product decision that still needs an explicit answer.

REALITY CHECK

The extreme horde targets and cinematic visual goals are feasible only through tiered simulation, aggressive batching, hidden low-cost representations, capability scaling, and continuous profiling. The architecture must support ambition without pretending that every entity can receive maximum fidelity at all times.

Contents

1. Vision and product position

ONE-SENTENCE PITCH

A high-fidelity isometric zombie survival game in which an authored city sits above a hidden destructible structural grid, allowing the player and the horde to permanently reshape routes, defenses, visibility, acoustics, and shelter.

The game inherits the strategic readability of a survival diorama without copying Project Zomboid's retro-isometric surface treatment. The visual target is modern, cinematic, graphic survival realism: grounded materials and lighting, painterly value control, selective strong outlines, readable silhouettes, amplified gore, and dense environmental storytelling.

Product identity

- Genre: single-player isometric urban zombie survival, systemic sandbox, and directed long-form campaign hybrid.
- Platform: desktop browser first, with capability-scaled mobile support where WebGPU and device limits make it viable.
- Production target: a complete indie game and a sufficiently explicit blueprint for coding-agent implementation.
- Primary differentiator: the environment is not a static backdrop. Existing spaces can be breached, reinforced, burned, dismantled, obstructed, and repurposed.
- Scale differentiator: the world supports individual close combat and very large hordes through multiple simulation and rendering representations.

What the game is not

- Not a direct Project Zomboid clone.
- Not a permanently voxel-looking world.
- Not a conventional wave-defense game with a scripted attack every night.
- Not a physics sandbox where every prop, body, and fragment remains a full rigid body forever.
- Not a React component tree representing the simulated world.
- Not a promise of identical visual quality or horde capacity on desktop and mobile.

Recommended market-facing description

A cinematic survival diorama where every wall is a decision. Break through an apartment, seal a stairwell, create a firing slit, reroute a horde, or destroy the shelter you meant to save. The city remembers what you change.

2. Experience pillars and player loop

Design pillars

Pillar	Meaning in play
The city remembers	Doors, windows, breaches, blood, fire, corpses, moved objects, and depleted resources persist as world state.
Noise is the real enemy	Sound, light, alarms, impact, and crowd agitation create delayed consequences beyond immediate combat.
Shelter is temporary	A safehouse can be improved but never made permanently invulnerable. Routes, supplies, fire, and pressure change its viability.
Preparation beats statistics	Observation, equipment, exits, timing, and spatial planning matter more than raw character levels.
Every opening is a tradeoff	A breach can create escape, sight, airflow, noise, exposure, or a horde route at the same time.
Hordes are geography	A large zombie mass is not merely a combat encounter. It changes which streets, buildings, and districts are usable.

Core loop

1. Observe the block, weather, light, zombie density, alarms, possible shelters, and escape routes.

2. Prepare a loadout based on a purpose, not on maximum carrying capacity.
3. Enter and scavenge a target structure while managing noise, visibility, time, and fatigue.
4. Modify the environment to create access, cover, concealment, or delay.
5. Create consequences through broken glass, gunfire, fire, moving furniture, light, and disturbed populations.
6. Return, recover, repair, sort, treat injuries, and decide whether the current shelter remains viable.
7. Respond to migration, depletion, weather, structural damage, contamination, or a compromised route.

Recommended campaign shape

Use a hybrid structure: the world supports open-ended survival, while a medium-term objective creates direction. The objective can evolve through radio repair, evacuation rumors, a missing person, preparation for winter, or a district-wide emergency. The player may ignore it temporarily, but it prevents the sandbox from becoming aimless.

TENSION RULE

Every major system should feed the same question: how safe is this place now, and what did the player do to change that answer?

3. World, survival, and progression

Initial complete-game scope

- One dense city with multiple districts and seamless streets, interiors, rooftops, and selected underground spaces.
- One primary survivor at a time, with rare human encounters and radio voices before full companion systems.
- Long-form survival with save persistence, district migration, and escalating resource pressure.
- Multiple shelter candidates rather than a single fixed base slot.
- Contextual crafting, repairs, barricading, controlled furniture movement, traps, and utilities.
- Mostly grounded infected archetypes with behavioral and anatomical variation rather than fantasy boss monsters as the default.

Survival systems

Launch system	Purpose	Depth rule
Hunger and thirst	Create expedition planning and shelter logistics.	Slow pressure, not constant meter babysitting.
Fatigue and sleep	Force safe windows and risk assessment.	Sleep quality depends on security, pain, noise, and temperature.
Bleeding and pain	Make combat consequences persist.	Clear treatment choices and readable severity.
Infection risk	Support dread without hiding every outcome.	Rules must be consistent and communicated through symptoms.
Encumbrance	Make loot selection meaningful.	Use weight, container capacity, and quick-access cost rather than spreadsheet complexity.
Stress and panic	Connect perception, crowds, darkness, and competence.	Affect control and awareness without removing agency.

Inventory and crafting philosophy

- Inventory is container-based: clothing, backpacks, cupboards, trunks, shelves, crates, and floor piles are explicit containers.
- Weight and capacity matter, but routine sorting should support fast transfer and category rules.
- Quick-access slots have animation and timing consequences during danger.
- Crafting is primarily common-sense and contextual. Tools, materials, skill, and a target object expose valid actions.
- Recipes are reserved for non-obvious processes, specialist equipment, chemistry, medicine, or advanced fabrication.
- Repairs and modifications should reuse the same material and tool logic as destruction wherever possible.

Progression

Progression should increase competence, options, and reliability rather than turning the survivor into a superhuman damage dealer. Skills improve speed, noise, waste, durability, treatment quality, structural knowledge, firearm handling, and the ability to interpret world information. Equipment, safe routes, cached supplies, and learned geography are at least as important as character stats.

4. Environment modification and destruction

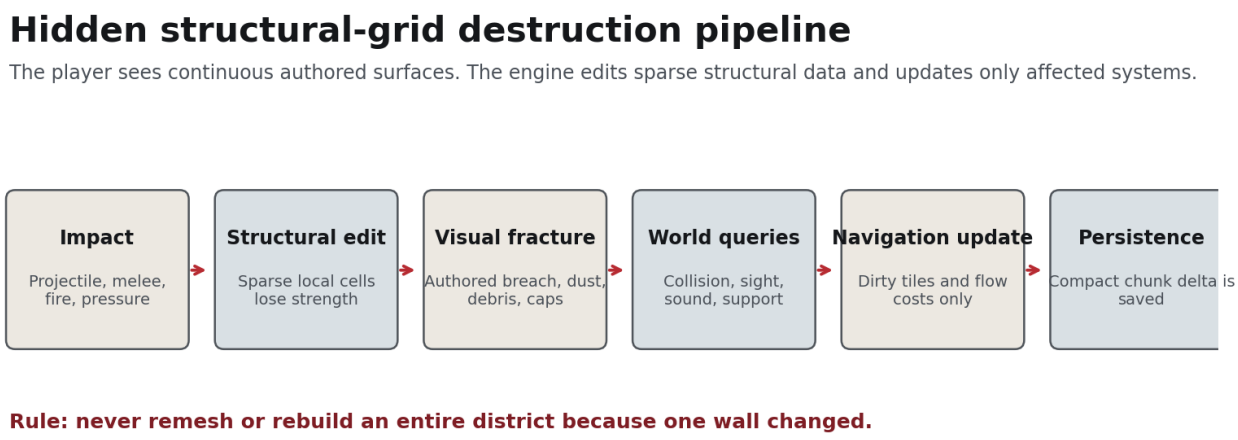

Figure 1. Localized destruction updates the visual world and only the affected simulation systems.

Core architectural choice

LOCKED DIRECTION

Use authored continuous geometry and materials for the visible world. Place a sparse, hidden structural grid inside destructible modules. The player does not see cubes unless a later tool or mode deliberately exposes the structure.

A globally dense voxel world would make authored materials, image-to-3D assets, lighting, UVs, navigation, and memory more difficult while encouraging a block-like visual result. The hybrid system preserves visual quality and still gives destruction systemic consequences.

Structural module model

```
StructuralModule
  stable module identifier
  local transform and bounds
  authored intact mesh references
  sparse occupancy cells
  material and strength per occupied cell
  support graph and anchor points
  openings, doors, windows, and service routes
  fracture families and breach thresholds
  collision proxy state
  navigation obstruction state
  acoustic and visibility state
  persistent modification delta
```

Supported modification classes

Class	Examples	Simulation consequences
Surface damage	Bullet holes, cracks, chipped plaster, broken glass.	Visual damage, sound, limited penetration changes.
Functional modification	Lock, unlock, close, board, reinforce, weld, brace.	Access time, strength, noise, visibility, path cost.
Breach creation	Break a wall section, cut a shutter, remove boards.	New portal, sight line, acoustic connection, horde route.
Obstruction	Move furniture, pile debris, park a vehicle.	Local flow cost, collision, climb or push behavior.
Support damage	Destroy stairs, beams, floors, or columns.	Local collapse risk and route deletion.
Fire and heat	Ignite wood, fuel, fabric, vegetation.	Damage over time, light, smoke, sound, spread, evacuation.
Utility work	Restore power, cut circuits, route water, silence alarms.	Lighting, noise, crafting, refrigeration, hazards.

Visual destruction rules

- Use authored fracture families, local damage masks, decals, interior material layers, wound-like edge caps, and pooled debris.
- Irregular holes must visually hide the structural cell shape.
- Debris can begin as physics objects and settle into cheap static or instanced representations.
- Persistent rubble is represented by compact state, not by keeping thousands of active rigid bodies.
- The same breach state informs rendering, collision, pathing, light, sound, fire, saving, and AI decisions.

Scope guardrails

- Launch with destructible doors, windows, selected walls, selected floors, barricades, furniture obstacles, and local fire.
- Treat full multi-building collapse, terrain excavation, and arbitrary civil-engineering construction as later milestones.
- Do not allow a small structural edit to trigger a full district remesh or a complete navigation rebuild.

5. Zombie ecology, hordes, and behavior

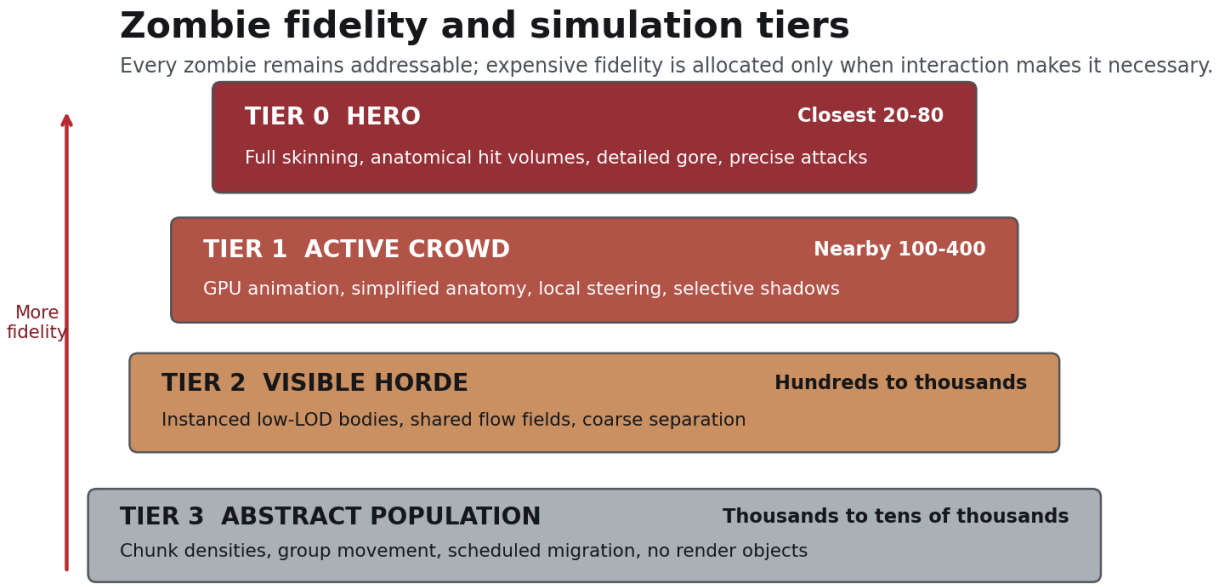

Figure 2. Fidelity is a dynamic resource. Interaction promotes individual zombies into more expensive tiers.

Authoritative principle

LOCKED ARCHITECTURE

A zombie is compact simulation data first. A Three.js object, skinned mesh, physics body, or React component is only an optional representation.

Tier transitions

Tier assignment depends on distance, visibility, threat, camera importance, target status, recent damage, current attack, and available performance budget. Promotion and demotion must preserve identity, health, anatomy, equipment, and behavior state. A targeted distant zombie can be promoted before the final hit response is resolved.

Population representation

Tier	Representation	Update model	Typical role
0 Hero	Individual entity plus detailed visual actor.	High-frequency perception, anatomy, attacks, animation.	Direct combat and close inspection.
1 Active crowd	Individual entity with batched visual representation.	Reduced-frequency perception and shared navigation.	Nearby pressure and interactive crowd.
2 Visible horde	Individual addressable IDs with heavily batched rendering.	Shared fields, coarse avoidance, sparse decisions.	Mass spectacle and ranged interaction.
3 Abstract population	Chunk-level groups, densities, and migration vectors.	Scheduled or event-driven updates.	Offscreen world ecology.

Behavior model

- Perception is stimulus-driven: sound, sight, light, movement, nearby agitation, fire, and scent-like residual cues if later desired.
- Zombies do not receive omniscient player coordinates.
- Individual behavior can be a compact state machine or utility score operating at tier-appropriate frequencies.
- A horde has group intent, density, momentum, and attraction without becoming one inseparable entity.
- Doorways and narrow routes permit compression, pushing, climbing, piling, and controlled overlap rather than demanding perfect non-overlap.
- World damage can become a group action: repeated pressure weakens barricades, doors, and selected structures.

Archetype composition

```
ZombieArchetype
  body and skeleton family
  locomotion profile
  perception profile
  attack profile
  anatomy and sever rules
  durability and armor profile
  animation set
  clothing and material family
  audio set
  special behavior modifiers
  allowed simulation and render tiers
```

Different zombie types should be composed from data rather than separate hardcoded classes. Examples include shamblers, fresh infected, crawlers, armored emergency personnel, decomposed fragile bodies, agitated runners, burned zombies, and partially dismembered variants.

6. Combat, hit detection, gore, and dismemberment

Combat intent

Combat is dangerous, readable, and spatial. One or two zombies can be managed through timing and positioning. Groups create containment and escape problems. Firearms solve immediate threats while creating sound, ammunition, penetration, and line-of-fire consequences.

Individual targeting at horde scale

1. A shot or melee sweep queries chunk-local spatial acceleration structures.
2. Candidate zombies are gathered from only intersected cells or bounds.
3. Candidates are ordered by projectile travel and filtered by tier-appropriate hit geometry.
4. The selected target is promoted if the final response requires detailed anatomy or animation.
5. Damage resolves against a named anatomical region, armor, penetration, and current posture.
6. The authoritative zombie entity records health, severed parts, locomotion changes, and death state.
7. Rendering receives a compact event for hit reaction, particles, wound visuals, and detached parts.

Hit-volume tiers

Context	Hit representation	Purpose
Hero combat	Head, neck, torso sections, upper and lower limbs.	Accurate location, armor, severing, and reactions.
Active crowd	Head, torso, left/right arms, left/right legs.	Individual ranged hits at lower cost.
Distant horde	Head sphere plus body capsule, or coarse projected bounds.	Maintain targetability before promotion.
Player attack contact	Timed attack volumes tied to animation windows.	Prevent damage from mere navigation overlap.

Dismemberment contract

- Use modular anatomical segmentation and wound-cap geometry, not arbitrary runtime mesh cutting for ordinary combat.
- Each detachable region has bone ownership, render ownership, sever thresholds, collision rules, detached-part assets, and behavior consequences.
- Missing limbs change attacks, locomotion, balance, crawling, reach, and threat level.
- Detached limbs are pooled and become cheap settled props after a short active window.
- Ordinary head destruction is fatal unless an archetype explicitly overrides it.

Gore art direction

- Physically grounded materials with graphically amplified shapes and timing.
- Directional blood spray and mist indicate impact direction and weapon energy.
- Large readable stains and sever silhouettes matter more than microscopic simulation at normal camera distance.
- Close combat can show wet material response, wound geometry, and transferred blood.
- Distant horde gore uses pooled simplified effects and sparse hero moments.

7. Art direction and cinematic isometric presentation

VISUAL DEFINITION

Graphic survival realism presented as a cinematic isometric diorama: Project Zomboid-style spatial readability, modern physically grounded lighting and materials, Left 4 Dead-style action clarity, The Last of Us-style environmental color and decay, and selective graphic-novel outlines and impact language.

Style balance

Source quality	Use	Do not copy
Project Zomboid	Readable cutaway spaces, functional clutter, city-scale survival grammar, small-character legibility.	Retro tile and sprite appearance as the final surface style.
Left 4 Dead	Bold infected silhouettes, readable combat effects, strong color accents, crowd immediacy.	First-person composition or arcade pacing by default.
The Last of Us	Decay, vegetation, dampness, light storytelling, material richness, color scripting.	Close-camera asset density applied indiscriminately to an isometric view.
Graphic novels	Selective outlines, compressed values, impact shapes, authored contrast.	Uniform black outlines on every distant object and horde member.

Small-character readability rules

- Evaluate every character at expected gameplay pixel height before approving close-up detail.
- Prioritize gait, posture, head shape, limb position, clothing masses, weapon silhouette, and injury state.
- Use broad value groups rather than noisy high-frequency texture detail.
- Allow restrained exaggeration in head, hand, weapon, and limb motion where it improves readability.
- Hit reactions need clean directional motion and temporary posture breaks, not subtle mocap-only responses.

Outline hierarchy

Subject	Outline policy
Player	Stable strongest screen-space silhouette, subtle rim light, clear weapon shape.
Nearby threats	Medium outlines with head and limb separation and visible wound states.
Distant horde	Few or no per-body outlines. Treat the group as a moving dark mass with selective highlighted members.
Architecture	Restrained edge enhancement focused on doors, windows, stairs, breach edges, and interactable structure.
Decorative clutter	Minimal edge treatment to avoid noise.

District color scripts

District	Base palette	Accents
Residential	Faded warm interiors, damp greens, grey daylight.	Domestic pastels, emergency red, warm lamps.
Hospital	Cold cyan, stained white, desaturated steel.	Medical blue, alarm red, fluorescent green.
Industrial	Rust, soot, charcoal, dull metal.	Sodium orange, hazard yellow, furnace glow.
Commercial	Wet asphalt, dark glass, dead advertising.	Broken neon, retail color fragments, police blue.
Suburban overgrowth	Washed pastel siding, vegetation, pale concrete.	Flowers, toys, bright abandoned objects.
Burn zone	Ash, blackened structure, desaturated haze.	Ember orange, hot white, warning paint.

Material philosophy

Use PBR as the physical base, then art-direct roughness, saturation, microdetail, and value range so the image reads at distance. Generated photoreal textures must be cleaned, normalized, and simplified. Material response should distinguish wetness, blood, dust, char, glass, metal, plastic, cloth, and organic matter without producing uncontrolled sparkle or visual noise.

8. Camera, visibility, and building cutaways

Recommended camera

- Near-orthographic perspective rather than mathematically perfect orthographic projection.
- Approximately 35 to 45 degrees downward pitch with a diagonal city-friendly yaw.
- Limited tactical zoom range, slightly closer indoors and able to pull back for major horde visibility.
- 90-degree rotation steps as the default for spatial predictability, with free rotation considered only after environment readability tests.
- Stable combat framing. Avoid automatic cinematic movement that changes aiming geometry during ordinary play.

Visibility system

- Hide or fade roofs and upper wall sections using room, portal, and camera-occlusion logic.
- Preserve enough wall base and structure to communicate enclosure and breach state.
- Use interior darkness and line of sight as gameplay systems, not only cosmetic post-processing.
- Do not make all interiors visible simply because the camera is above them.
- Provide a clear visual language for known, currently visible, heard, and remembered threats.

Aiming and selection

Mouse aiming can use a world-plane cursor with assisted depth resolution and target confidence. Controller aiming can use directional intent, soft lock, and threat cycling. The interface must distinguish selected target, projected impact, obstruction, penetration uncertainty, and friendly or structural risk without turning combat into a tactical overlay.

9. User interface, controls, and accessibility

Interface character

The interface should feel like practical survival equipment expressed through a sleek modern game UI. It can borrow visual cues from a notebook, watch, radio, map, medical kit, and tool markings without slowing interaction or disguising essential information.

HUD principles

- Show only information that changes an immediate decision.
- Use world-space prompts sparingly and anchor them to stable interaction targets.
- Do not make survival meters compete continuously with the environment.
- Provide explicit status detail in an inspectable health and condition panel.
- Use animation and sound to communicate state before adding text labels.

Input plan

Platform	Recommended model
Desktop keyboard and mouse	WASD movement, mouse aim, contextual interaction, radial or quick action access, direct inventory manipulation.
Desktop controller	Twin-stick movement and aim, target assistance, radial actions, container transfer shortcuts.
Mobile capability tier	Virtual movement and aim zones, context-sensitive action button, aggressive auto-target assistance, simplified inventory interactions.

Accessibility requirements

- Full input remapping and separate sensitivity controls.
- Outline strength, target highlight, gore intensity, camera shake, flashes, and motion reduction settings.
- Color-independent interaction and damage indicators.
- Scalable UI and readable high-contrast text modes.
- Optional pause or slowdown for inventory and complex contextual actions in single-player.
- Audio cue subtitles or visual indicators for alarms, breaking glass, and directional threats.

10. Technology stack and architecture boundaries

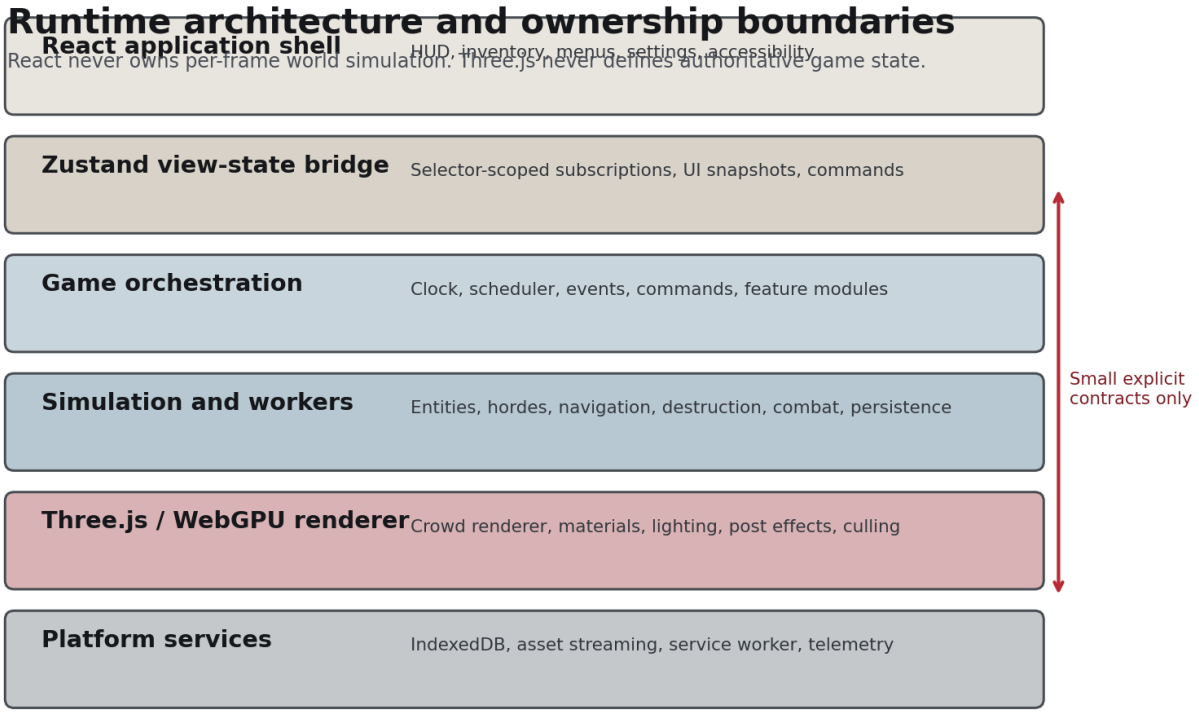

Figure 3. Ownership boundaries keep React, simulation, rendering, and platform services independently optimizable.

Locked stack

```
Three.js
React
TypeScript
Zustand
Vite
WebGPU-first rendering through Three.js
Web Workers
SharedArrayBuffer where deployment isolation permits
IndexedDB for structured local persistence
Optional WebAssembly only for measured bottlenecks
```

Renderer integration

Use a direct Three.js engine hosted by React. React Three Fiber may be used only for limited high-level composition if profiling supports it, but the crowd, streaming world, destruction, animation buffers, and per-frame simulation must not be represented as React component trees.

Architecture boundaries

Owner	Owns	Must not own
React	Application shell, panels, HUD, menus, accessibility, loading and error boundaries.	Per-frame entities, projectile state, crowd movement, animation time.
Zustand	UI view state, settings, selected snapshots, commands, session state.	Authoritative world arrays or thousands of changing transforms.
Simulation	Authoritative game state, fixed ticks, AI, combat, destruction, navigation requests.	DOM and Three.js objects.
Renderer	Visual proxies, GPU buffers, lights, effects, interpolation, culling.	Gameplay authority or save semantics.
Workers	Heavy batch tasks and asynchronous world processing.	Direct DOM or React manipulation.

Fixed time and interpolation

The authoritative simulation runs at a fixed tick independent of render rate. Rendering interpolates between stable snapshots. Different subsystems can use lower update frequencies through the scheduler, but the order of authoritative changes must remain explicit and testable.

11. React and Zustand engineering rules

Store strategy

Use multiple purpose-specific stores or a carefully composed slice store for application state. The simulation remains outside Zustand and publishes deliberately small view snapshots. Selector-scoped subscriptions are mandatory for frequently changing state.

```
Recommended stores
  session store
  UI and modal store
  player-view store
  inventory-view store
  crafting-view store
  map-view store
  settings store
  input store
  diagnostics store

Not a store
  zombie positions
  projectile transforms
  animation clocks
  chunk entity arrays
  collision contacts
  structural cell arrays
```

Mandatory Zustand practices

- Every React component subscribes to the smallest practical selector.
- Never call a store hook without a selector in production UI code.
- Prefer primitive selectors. Use shallow comparison for small tuples or objects only when appropriate.
- Use subscribeWithSelector for engine-to-store and non-React subscriptions.
- Use slices to group state and actions by domain while keeping a clear dependency direction.
- Apply persistence only to explicit settings or session slices, not to transient engine state.
- Keep actions stable and avoid rebuilding command objects in render paths.
- Throttle or event-gate high-frequency UI snapshots such as health interpolation or cursor targeting.

Anti-patterns

Do not	Reason
Render one React component per zombie.	Reconciliation and subscription overhead become architectural bottlenecks.
Write every player transform to Zustand every animation frame.	It creates unnecessary subscriptions and UI work.
Store Three.js objects in broad UI stores.	It mixes lifecycle, serialization, and authority boundaries.
Use a single catch-all selector for a full store.	Unrelated updates trigger unnecessary rendering.
Let components mutate simulation data directly.	Commands and validation become impossible to trace or replay.

React update contract

The UI issues commands such as equip, craft, move item, confirm action, change setting, or select target. The engine validates the command and later publishes a view-state result. A component does not directly edit inventory arrays, world entities, or structural cells.

12. Simulation model and data ownership

Data-oriented core

Use compact typed arrays or similarly cache-friendly structures for high-count entities. The architecture may borrow ECS principles without requiring a fashionable general-purpose ECS library. Optimize around actual access patterns: position, velocity, state, target, health, tier, chunk, and compact flags.

```
High-count zombie data groups
  identity and archetype
  position, heading, velocity
  current state and timers
  health and anatomical flags
  target and stimulus references
  chunk, spatial cell, and navigation group
  simulation tier and render tier
  animation state and phase

Low-count side data
  unique equipment
  active wounds
  named quest state
  detailed recent-event history
```

System scheduling

Frequency	Examples
Every fixed tick	Movement integration, close combat, projectile resolution, immediate collision.
Every 2-4 ticks	Nearby perception, local steering, attack decision refresh.
Every 5-15 ticks	Distant perception, horde cohesion, shelter pressure, some status effects.
Every second or event	Abstract population migration, district supply pressure, long timers.
On demand	Path requests, chunk activation, hit promotion, structural edits, save serialization.

Event and command design

- Commands express player or UI intent and can fail with explicit reasons.
- Events describe facts that already occurred and can feed rendering, audio, UI, analytics, and persistence.
- Avoid a global event bus with untyped arbitrary strings.
- Use bounded queues and pooled event records for high-frequency gameplay events.
- Separate ephemeral visual events from persistent world mutations.

13. World streaming, chunking, and persistence

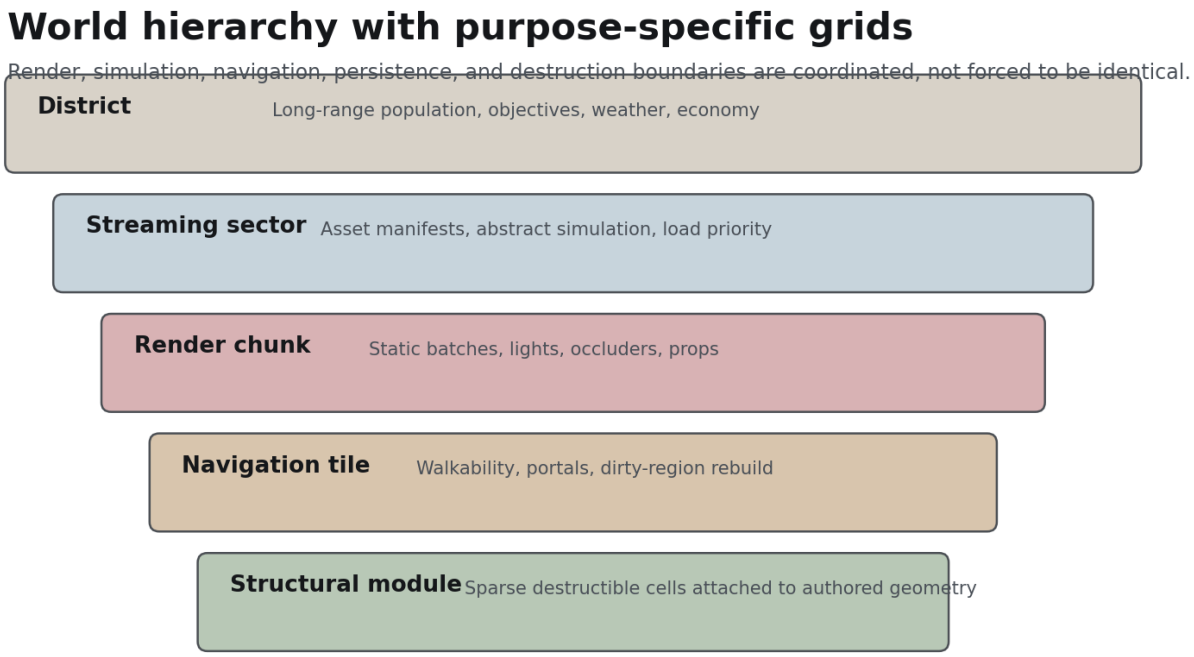

Figure 4. Different systems need different spatial boundaries; they should align where useful without being forced into one universal chunk size.

Proposed spatial hierarchy

Layer	Provisional scale	Responsibility
District	Approximately 512 m square	Long-range population, objectives, weather, resource economy, save partition.
Streaming sector	Approximately 128 m square	Asset manifest, activation priority, abstract simulation boundary.
Render chunk	Approximately 32 m square	Static batches, visibility, lights, decals, scene attachment.
Navigation tile	Approximately 16 m square	Local path data and dirty-region rebuild.
Structural module	Object-local and sparse	Destruction cells for walls, floors, doors, supports, and selected props.

All dimensions are defaults for prototyping, not hardcoded truths. They belong in centralized configuration and must be validated against camera distance, building scale, load latency, nav rebuild cost, and device memory.

Chunk lifecycle

```
unloaded
  -> abstract simulation only
  -> metadata and dependencies requested
  -> CPU data loaded
  -> simulation active
  -> visually represented
  -> high-detail active
  -> cooling down
  -> persisted and evicted
```

Streaming priority

- Player movement and projected movement direction.

- Camera visibility and zoom.
- Nearby active hordes and their velocity.
- Audio propagation and important offscreen events.
- Objective relevance and predicted doorway or stair transitions.
- Available CPU, GPU, memory, and network or storage bandwidth.

Persistence model

Store immutable base world packages separately from compact modification deltas. A save records doors, breaches, barricades, moved persistent objects, searched containers, local population changes, corpses, fires, utilities, dropped items, and mission state. It should not duplicate every untouched building asset.

- Use schema versions and explicit migrations from the first public build.
- Partition saves by district or sector so one corrupted record does not destroy the entire world.
- Serialize in a worker and write to IndexedDB asynchronously.
- Use periodic checkpoints plus a short mutation journal for crash recovery.
- Validate asset and world-version compatibility before loading.

14. Navigation, collision, and spatial queries

Hybrid navigation stack

Layer	Use
Global region and portal graph	Long-distance route selection across districts, blocks, floors, staircases, doors, windows, alleys, and roofs.
Tiled precision navmesh	Player, human NPCs, hero zombies, complex interiors, exact short-range traversal.
Cost and flow fields	Large groups moving toward shared stimuli, breaches, exits, or the player.
Local steering grid	Separation, wall avoidance, pressure, queueing, pushing, and group texture.
Abstract migration graph	Offscreen horde movement between sectors and districts.

Why not one path per zombie

Thousands of independent A* searches and continuously repaired path corridors would turn shared movement into duplicated work. Large groups should share target fields and corridor intent. Individual precision is introduced only near complex traversal or direct interaction.

Collision broad phase

- Use chunk-local uniform grids or spatial hashes for dynamic agents.
- Inspect only the entity cell and bounded neighboring cells.
- Keep collision layers explicit: movement, projectile, attack, interaction, sight, and audio may use different proxies.
- Represent ordinary moving zombies as circles on the ground plane plus vertical bounds.
- Promote to body capsules and anatomical volumes only when needed.

Static environment queries

- Separate visual mesh, collision proxy, navigation data, interaction targets, and destruction structure.
- Use simple boxes, capsules, convex forms, and occupancy fields for movement.
- Use chunk-local triangle acceleration structures only for precise ray or projectile queries.
- Mark only affected navigation tiles and cost cells dirty after destruction.
- Cache shared flow fields by target, movement profile, and navigation revision.

Crowd pressure and doorways

Perfect collision resolution is neither necessary nor desirable for a compressed horde. Lower tiers may overlap slightly, use density pressure, and resolve separation gradually. Close bodies receive stronger correction. Doorways can support queueing, climbing, pushing, grabbing, and structural pressure without asking every agent to find a unique collision-free route.

15. Rendering, lighting, materials, and effects

WebGPU-first policy

Use Three.js WebGPURenderer as the primary renderer and design capability checks around actual adapter limits and features. Maintain a reduced compatibility path only when it does not distort the core architecture. High-quality desktop mode is the reference experience.

Crowd rendering paths

Path	Technique	Use
Hero actor	Conventional skinned mesh, full material set, detailed animation and wounds.	Closest and directly interacting zombies.
Instanced animated crowd	Shared mesh families, GPU-readable animation data, per-instance variation buffers.	Nearby and mid-distance interactive crowd.
Horde LOD	Reduced skeleton or vertex animation, aggressive geometry and material batching.	Hundreds or thousands of visible bodies.
Impostor or cluster	Far-distance image or simplified cluster representation.	Extreme distance and horde vistas.

Batching and variation

- Use instancing when geometry and material families match.
- Use batched drawing when multiple geometries share material families.
- Group materials into controlled families and texture arrays or atlases where practical.
- Generate variety through body, head, hair, clothing modules, masks, palette, dirt, blood, posture, scale, and animation phase.
- Do not create a unique shader and material graph for every generated zombie.

Lighting stack

- Directional sun or moon with distance-managed shadow cascades or an equivalent budgeted system.
- Baked or precomputed indirect lighting for static architecture where destruction rules allow it.
- Dynamic local lights for flashlights, fire, alarms, vehicles, and gameplay-critical sources.
- Contact and ambient occlusion concentrated near the player and important structure.
- Atmospheric fog, weather extinction, and interior exposure transitions.
- Shadow casting prioritized by screen contribution, tier, distance, and threat.

Post-processing

Effect	Rule
Anti-aliasing and reconstruction	Stable outlines and thin geometry are more important than raw sharpness.
Color grading	District, time, weather, and danger state use authored profiles.
Bloom	Selective practical light and wet highlight support, never a universal haze.
Ambient occlusion	Improve diorama grounding without dirtying every surface.
Depth and fog	Separate horde layers and city depth while preserving target readability.
Dynamic resolution	Respond to GPU pressure before dropping simulation correctness.
Damage feedback	Use camera shake, vignette, blur, and chromatic effects sparingly and accessibly.

Resource lifecycle

Every streamed geometry, texture, material, render target, buffer, and effect must have explicit ownership and disposal. The diagnostics layer must show GPU estimates, live draw calls, triangles, active lights, shadow casters, texture residency, instance counts, and streaming pressure.

16. Generated 3D asset pipeline

Asset-pipeline principle

LOCKED RULE

Image-to-3D output is source material, not a shippable runtime asset. Every model passes through normalization, validation, optimization, and style control.

Zombie asset contract

Requirement	Description
Scale and axes	Consistent units, orientation, origin, floor contact, and forward direction.
Skeleton family	Approved skeleton, bone names, limits, bind pose, and animation compatibility.
Anatomical regions	Named render sections and bone ownership for hit detection and dismemberment.
LOD chain	Hero, crowd, horde, and optional impostor representations.
Material family	Limited shader families with validated maps, ranges, and texture resolution.
Texture delivery	KTX2/Basis compressed textures with mipmaps and channel packing where useful.
Geometry delivery	glTF/GLB, optimized indices, compression, predictable primitive and material counts.
Collision	Ground footprint, body capsule, head and anatomical proxies.
Performance metadata	Triangle counts, texture memory, bones, draw groups, expected tiers.

Pipeline stages

1. Import generated source and record provenance and license status.
2. Clean topology, remove hidden garbage, close problematic surfaces, and normalize scale.
3. Retopologize or decimate for the hero target while preserving silhouette.
4. Assign an approved skeleton and validate weights and deformation.
5. Split anatomical regions and create wound caps and detached-part assets.
6. Normalize UVs and bake source appearance into controlled material channels.
7. Apply art-direction pass to reduce noise and align palette and roughness.
8. Generate LODs, shadow proxy, collision proxies, and optional impostors.
9. Compress geometry and textures, generate metadata, and run automated validation.
10. Test at normal gameplay distance, in a crowd, under representative lighting, and during gore states.

Environment assets

- Architecture should be modular enough for streaming and cutaways without looking kit-built.
- Destructible surfaces require intact mesh, interior layers, fracture families, structural mapping, and collision states.
- Clutter needs clear interaction categories and LOD or batching rules.
- Generated props must not introduce uncontrolled material counts or texture sizes.
- Every asset bundle carries dependencies and can be loaded and disposed independently.

17. Audio and sensory simulation

Audio as gameplay

Sound is both an aesthetic layer and a simulation input. The same event that the player hears can produce a stimulus with intensity, frequency character, duration, source type, obstruction, and propagation history.

Sound class	Gameplay use
Impacts and breaches	Attract zombies, reveal material, communicate structural damage.
Glass and alarms	Create sharp long-range stimuli and location-specific danger.
Gunfire	Immediate damage plus district-scale delayed consequences.
Footsteps and movement	Surface-dependent stealth feedback and nearby detection.
Zombie vocalization	Local agitation propagation and horde state communication.
Weather and machinery	Mask or amplify other sounds, change player confidence.

Propagation model

- Use a coarse sector graph for long-range sound spread.
- Use doors, windows, breaches, floors, and material properties as attenuation links.
- Reserve expensive ray or portal refinement for the active area.
- Represent a major event as a persistent disturbance that can continue influencing migration after the sample ends.
- Do not play individual zombie vocalization for every member of a large horde. Use layered group beds plus selected foreground voices.

18. Performance budgets and benchmark scenarios

MEASUREMENT RULE

Horde counts are benchmark gates, not marketing claims. Every target must specify hardware, resolution, camera, effects, active systems, and acceptable frame-time percentile.

Provisional desktop reference targets

Metric	Normal gameplay target	Extreme benchmark target
Frame rate	60 FPS at 1440p on reference dedicated GPU.	At least 45 FPS during the defined maximum horde stress scene.
Visible zombies	300-800 depending on scene.	2,000 individually addressable low-tier bodies, with stretch goal beyond.
Detailed hero zombies	20-40.	Up to 80 under constrained scene conditions.
Active individual simulation	500-1,500.	Up to 5,000 tiered entities in loaded sectors.
Abstract world population	Tens of thousands.	Limited primarily by save and district simulation design.
Draw calls	Budgeted by scene and material family.	Must remain stable as crowd count rises through batching.
Main-thread CPU	Prefer below 5 ms average.	No sustained long tasks that disrupt input or UI.
GPU frame	Prefer below 12 ms at target quality.	Dynamic resolution and effect scaling engage before failure.

These numbers are intentionally ambitious and must be revised after the first GPU-animation, destruction, and navigation prototypes. The architecture should make scaling possible, while production planning must follow measured results.

Benchmark scenes

Scene	Required stress
Crowd avenue	Long street, thousands of visible zombies, weather, outlines, shadows, gunfire, and camera rotation.
Breach cascade	A horde breaks multiple barricades and walls while dirty navigation and visual fracture updates occur.
Dense interior	Multi-room building with cutaways, lights, clutter, close combat, gore, and many interaction targets.
Streaming sprint	Player moves quickly across sector boundaries while a horde crosses adjacent sectors.
Corpse accumulation	Hundreds or thousands of settled bodies and stains with repeated save and reload.
Mobile capability	Reduced-quality scene establishes realistic lower-tier counts and thermal behavior.

Profiling requirements

- Record median, 95th percentile, and 99th percentile frame times.
- Separate main-thread, worker, GPU, streaming, garbage collection, and save costs.
- Maintain automated performance captures for benchmark scenes in continuous integration where practical.
- Treat memory growth and resource leaks as release-blocking defects.
- Add debug controls to freeze tiers, force LODs, show spatial grids, visualize flow fields, and inspect dirty navigation tiles.

19. Quality tiers, desktop, and mobile

Capability detection

Select a quality tier from measured startup tests and GPU adapter limits rather than browser names alone. Store a user override, but protect against settings that exceed safe resource limits.

Tier	Experience
Desktop high	Reference lighting, high texture residency, large horde budgets, strong effects, more hero zombies, greater shadow distance.
Desktop medium	Reduced shadow and post effects, more aggressive LOD, smaller active radius, lower horde presentation budget.
Desktop compatibility	Reduced renderer features and crowd capacity; gameplay systems remain consistent where possible.
Mobile WebGPU	30 FPS target, lower internal resolution, simplified lighting, fewer hero bodies, smaller streaming radius, touch controls.

Scaling order

1. Reduce internal render resolution and expensive post-processing.
2. Reduce shadow distance, shadow resolution, and secondary shadow casters.
3. Reduce crowd animation fidelity and hero promotion budget.
4. Increase LOD aggressiveness and reduce texture residency.
5. Reduce debris, particles, persistent visual corpses, and dynamic local lights.
6. Reduce visible horde density only after visual costs have been addressed.
7. Never reduce authoritative combat correctness merely to hide a rendering problem.

Mobile promise

Mobile is a capability-scaled version of the same game, not guaranteed visual parity. Input, UI density, thermal limits, memory, storage, and horde scale require dedicated design. It should remain optional until the desktop vertical slice proves the architecture.

20. Codebase organization and configuration

Recommended project layout

```
src/
  app/                React shell, routing, lifecycle, error handling
  ui/                 HUD, panels, menus, accessibility, design system
  stores/             Zustand stores, slices, selectors, persistence adapters
  game/
    core/              clock, scheduler, events, commands, IDs
    simulation/        entity data and system execution
    world/             districts, sectors, chunks, portals, mutations
    zombie/            population, behavior, tiers, archetypes
```

```
    combat/             weapons, projectiles, damage, anatomy
    destruction/        structural modules, fracture, fire, support
    navigation/         graphs, navmesh, flow fields, steering
    inventory/          containers, items, crafting, equipment
    persistence/        snapshots, journals, migrations
  render/
    engine/             Three.js lifecycle and frame orchestration
    crowd/              GPU animation, instances, LOD, culling
    world/              chunk render proxies and cutaways
    materials/          approved material families and nodes
    lighting/           sun, local lights, shadows, probes
    effects/            weather, gore, post-processing
  workers/              nav, world, population, serialization workers
  assets/               manifests, registries, loaders, validators
  config/               typed values and quality profiles
  diagnostics/          profilers, overlays, logs, benchmark tools
  tests/                 unit, integration, replay, performance scenarios
  tools/                 import, baking, compression, content validation
```

Configuration rules

- No unexplained magic numbers in systems or materials.
- Every value has a unit, domain owner, default, valid range, and quality-tier behavior.
- Derived values are computed from a smaller set of meaningful source values.
- Content data and tuning values are separated from engine invariants.
- Configuration changes can be diffed, validated, and included in bug reports.
- Production builds reject invalid content rather than silently inventing fallbacks.

Config domains

```
game | world | streaming | time | player | survival | items
inventory | crafting | structures | destruction | fire | zombies
perception | hordes | navigation | collision | combat | weapons
camera | rendering | lighting | shadows | materials | postFX
weather | audio | UI | input | accessibility | saving | debug
```

Type and naming guidance

- Use explicit IDs for entities, chunks, assets, modules, stimuli, and commands.
- Avoid passing raw object references across worker or persistence boundaries.
- Use discriminated event and command types with exhaustive handling.
- Distinguish world meters, grid cells, pixels, seconds, ticks, degrees, and radians in names or types.
- Use ownership-oriented names such as SimulationZombie, ZombieRenderProxy, and ZombieViewSnapshot rather than one overloaded Zombie object.

21. Testing, diagnostics, and production operations

Test layers

Layer	Examples
Unit	Damage rules, item transfer, structural strength, tier transitions, config validation.
Deterministic simulation	Recorded command sequences produce expected authoritative outcomes.
Integration	Breach changes navigation, visibility, audio, rendering, and persistence consistently.
Content validation	Skeleton names, LODs, textures, materials, fracture mapping, and budgets.
Visual regression	Reference captures for outlines, cutaways, gore, weather, and material states.
Performance	Automated or repeatable benchmark scenes with stored baselines.

Layer	Examples
Save compatibility	Schema migration, partial corruption recovery, version mismatch behavior.

Required debug views

- Chunk, sector, district, and navigation-tile boundaries.
- Structural occupancy cells, support links, and dirty regions.
- Zombie tier, render tier, current state, target, and update frequency.
- Spatial hash occupancy and collision candidate counts.
- Flow-field vectors, path corridors, portals, and blocked links.
- Draw calls, triangles, instances, animation groups, lights, shadows, GPU memory estimates, and texture residency.
- Worker queue depth, simulation timing, frame timing, garbage collection, and save operations.

Failure behavior

- A failed asset should produce a clear placeholder and report, not a silent missing collider.
- Worker failure should degrade or restart a subsystem without corrupting authoritative state.
- Save writes should be atomic at the record level and retain the previous valid checkpoint.
- WebGPU device loss should present a controlled recovery path or explicit session-safe shutdown.
- Performance overload should lower quality through documented priorities instead of unpredictably dropping systems.

22. Roadmap and milestone gates

Milestone 0: Technical spikes

- GPU-instanced animated crowd with at least several hundred varied zombies.
- Target promotion from coarse crowd hit proxy to detailed anatomical response.
- Tiled navigation plus one shared flow-field prototype in a destructible test block.
- Sparse structural wall module that creates an irregular breach and updates collision and pathing.
- Chunk streaming with asset disposal and a basic save delta.
- React and Zustand shell proving selector-scoped UI updates without world-state subscriptions.

GATE 0

Do not build broad game content until the crowd, destruction, streaming, and state boundaries have demonstrated plausible performance and maintainability.

Milestone 1: Playable systems sandbox

- One city block with street, multi-room building, cutaway camera, loot, shelter actions, melee, firearm, sound attraction, and save/load.
- At least three zombie archetypes and anatomical damage.
- Basic day/night and weather lighting.
- One complete modify-defend-escape loop.

Milestone 2: Vertical slice

- One visually representative district with final-quality art direction.
- A beginning, medium-term objective, and a decisive horde event shaped by player modifications.
- Production asset pipeline, benchmark suite, accessibility pass, and stable save migration.
- Reference desktop quality target demonstrated on named hardware.

Milestone 3: Production foundation

- Multiple districts, content tools, automated validation, narrative framework, expanded survival and crafting.
- Population migration and long-term world simulation.
- Performance budgets enforced in content review.

- Mobile feasibility decision based on measured prototype, not aspiration.

Milestone 4: Complete indie game

- Complete campaign or hybrid objective structure, progression, endings, balanced survival economy, polished onboarding, and broad content variety.
- Stable platform support matrix, quality presets, crash recovery, save migrations, and release benchmark results.

23. Risks, tradeoffs, and non-goals

Major risks

Risk	Mitigation
Thousands of zombies plus individual anatomy	Tiered data, shared navigation, promotion on interaction, GPU animation, strict benchmark gates.
Destruction destabilizes navigation and saves	Sparse modules, local dirty updates, revisioned data, compact mutation journal.
Generated assets look inconsistent	Strict asset contract, art-direction pass, material families, automated validation.
AAA visual ambition overwhelms indie scope	Camera-aware detail, reusable systems, district color scripts, selective hero fidelity.
React leaks into engine hot paths	Ownership rules, direct Three.js engine, selector-only UI snapshots.
WebGPU capability fragmentation	Runtime limits, quality tiers, reduced compatibility path, explicit support matrix.
Mobile expands scope too early	Desktop-first milestone gates and a separate measured feasibility decision.

Launch non-goals unless separately approved

- Authoritative multiplayer or persistent online world.
- Every building fully collapsible at arbitrary granularity.
- Terrain excavation across the entire city.
- Full rigid-body persistence for every corpse and fragment.
- Equal desktop and mobile horde capacity.
- Unbounded procedural city generation replacing authored district design.
- Companion community simulation comparable to a colony-management game.
- Photoreal close-up fidelity for every crowd member.

Decision rule

When a feature conflicts with readability, systemic consequence, stable performance, or production capacity, preserve the central promise first: physically altering the city must meaningfully change survival and horde behavior.

24. Coding-agent directive and acceptance criteria

Implementation directive

AGENT INSTRUCTION

Build vertical systems that prove architecture contracts before expanding breadth. Every feature must identify its owner, authoritative data, render representation, worker boundary, configuration, persistence behavior, debug view, tests, and performance budget.

Non-negotiable engineering rules

1. Do not make React or Zustand authoritative for per-frame world state.

2. Do not create one Three.js object, animation mixer, physics body, path search, or React component per zombie by default.
3. Do not couple simulation records to visual objects.
4. Do not introduce magic numbers outside typed configuration.
5. Do not rebuild entire world regions for local destruction.
6. Do not merge navigation, collision, visual mesh, and interaction geometry into one representation.
7. Do not accept generated assets without validation and budgets.
8. Do not add a new high-cost effect without a quality-tier policy and benchmark scene.
9. Do not persist untouched base world data in every save.
10. Do not claim a horde count until a reproducible benchmark proves it.

Definition of done for a system

Area	Acceptance criteria
Function	Player-visible behavior matches the written rules and failure states are explicit.
Architecture	Ownership and data flow respect module boundaries.
Performance	Representative benchmark remains within the allocated budget.
Configuration	Values are centralized, typed, documented, and validated.
Persistence	Relevant state survives save, reload, migration, and partial failure.
Diagnostics	Debug view and timing information exist.
Testing	Unit and integration coverage includes edge cases and system interactions.
Accessibility	Relevant settings and alternative feedback are supported.

Feature template for future prompts

```
Feature name and player value
Design rules and failure states
Authoritative data and owner
Commands and events
Simulation frequency and tier behavior
Rendering representation and LOD
Collision, navigation, sound, and destruction interactions
React and Zustand view-state needs
Persistence and migration
Configuration values and units
Debug tools and telemetry
Tests and benchmark scenario
Explicit out-of-scope behavior
```

25. Locked decisions and open questions

Locked decisions

- Three.js, React, TypeScript, Zustand, and Vite.
- React for UI and application shell, direct Three.js engine for the world.
- Zustand selectors, shallow comparison where justified, subscribeWithSelector, and domain slices.
- Desktop browser first and WebGPU-first.
- Cinematic isometric or near-isometric diorama presentation.
- Graphic survival realism with selective strong outlines and explicit gore.
- Authored visible world with hidden sparse structural grids for destruction.
- Large horde architecture using tiered simulation and rendering.
- Multiple zombie types and individual hit reactions and dismemberment.
- Complete indie game as the production target.

Recommended defaults awaiting explicit confirmation

- Single-player only for the initial complete game.
- Hybrid sandbox plus medium-term objective structure.
- Mostly grounded zombie archetypes with a small number of exceptional behaviors.
- Near-orthographic camera with 90-degree rotation steps.
- Behavior-driven shelter pressure rather than a scripted nightly wave.
- Contextual crafting and modification of existing structures before freeform construction.
- World persistence through base packages plus compact deltas.

Open product decisions

Decision	Why it matters
Narrative objective and ending structure	Defines pacing, district progression, radio content, and replayability.
Death model	Traditional reload, persistent-world permadeath, or run structure changes save and emotional design.
Maximum destruction scope	Local breaches versus full support collapse changes tooling and production cost.
Minimum desktop hardware	Required to turn performance budgets into enforceable targets.
Official WebGL fallback policy	Determines renderer complexity and supported audience.
Camera rotation and zoom limits	Affects art production, occlusion, input, and combat readability.
Human NPC depth	Companions and communities multiply AI, narrative, animation, and persistence scope.
Mobile release commitment	Needs a separate input, UI, thermal, and content-scale plan.

Appendix A. Reference notes

The references below support the technology and comparative-art observations used in this handout. They do not substitute for project-specific prototypes and benchmark results.

Ref.	Source	URL
R1	Project Zomboid developer post: Build 41 characters rendered directly in isometric 3D space.	https://projectzomboid.com/blog/news/2019/10/streamzed-iii/
R2	Project Zomboid developer post: Build 41 animation and character overhaul.	https://projectzomboid.com/blog/news/2021/12/project-zomboid-build-41-released/
R3	Project Zomboid developer discussion of its tile-based visual pipeline and depth buffer.	https://projectzomboid.com/blog/news/2023/02/play-your-cardz-right/
R4	Three.js WebGPURenderer documentation.	https://threejs.org/docs/pages/WebGPURenderer.html
R5	Three.js InstancedMesh documentation.	https://threejs.org/docs/pages/InstancedMesh.html
R6	Three.js BatchedMesh documentation.	https://threejs.org/docs/pages/BatchedMesh.html
R7	Three.js GLTFLoader and KTX2Loader documentation.	https://threejs.org/docs/pages/GLTFLoader.html ; https://threejs.org/docs/pages/KTX2Loader.html
R8	Zustand selector, slices, and subscribeWithSelector guidance.	https://zustand.docs.pmnd.rs/learn/guides/beginner-typescript ; https://zustand.docs.pmnd.rs/learn/guides/slices-pattern ; https://zustand.docs.pmnd.rs/reference/middlewares/subscribe-with-selector
R9	Recast Navigation overview of tiled navmeshes and streaming.	https://recastnav.com/
R10	DetourCrowd documentation and proximity-grid based local steering.	https://recastnav.com/classdtCrowd.html
R11	Elijah Emerson, Crowd Pathfinding and Steering Using Flow Field Tiles.	https://www.gameaipro.com/GameAIPro/GameAIPro_Chapter23_Crowd_Pathfinding_and_Steering_Using_Flow_Field_Tiles.pdf
R12	MDN Web Workers API.	https://developer.mozilla.org/en-US/docs/Web/API/Web_Workers_API
R13	MDN SharedArrayBuffer requirements for secure, cross-origin isolated contexts.	https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/SharedArrayBuffer

Ref.	Source	URL
R14	MDN IndexedDB API.	https://developer.mozilla.org/en-US/docs/Web/API/IndexedDB_API
R15	MDN GPU adapter and device limits.	https://developer.mozilla.org/en-US/docs/Web/API/GPUAdapter/limits
R16	W3C WebGPU specification.	https://www.w3.org/TR/webgpu/
R17	Khronos KTX 2.0 and Basis Universal texture-delivery overview.	https://www.khronos.org/ktx/

Final guiding statement

BUILD THE ILLUSION, NOT THE COST

The player should experience a city full of individually vulnerable bodies and physically mutable spaces. The engine is free to use abstractions, tiers, batches, sparse grids, workers, and GPU techniques wherever those tricks remain invisible and the resulting behavior is consistent.

Immediate next workshop

The concept is sufficiently defined to begin architecture spikes. Before the vertical slice brief is finalized, the team should explicitly lock the following five decisions:

1. The campaign objective, ending structure, and role of long-term sandbox play.
2. The death and save model, including whether modified worlds survive character death.
3. The minimum desktop GPU and the official WebGPU or compatibility support policy.
4. The maximum launch scope of structural collapse, fire, and freeform obstruction.
5. The camera rotation, zoom limits, and primary keyboard, mouse, and controller control model.

FIRST PRODUCTION ACTION

Build one ugly but measurable test block containing a destructible wall, a multi-room interior, 500 or more animated zombies, shared horde navigation, a firearm with anatomical hits, save and reload, and a React HUD that never subscribes to per-frame world state.